CHAPTER 1

DESIGNING STACK OVERFLOW

1 Getting Ready: Stack Overflow 2
2 Requirements for Stack Overflow 4
3 Use Case Diagram for Stack Overflow 5
4 Class Diagram for Stack Overflow 10
5 Sequence Diagram for Stack Overflow 24
6 Activity Diagram for Stack Overflow 26
7 Code for Stack Overflow . . 28
8 Summary: Designing Stack Overflow 36
9 Key Insights: Stack Overflow 40

In this chapter, you will learn the design of Stack Overflow, including requirements, use case, class, sequence, activity diagrams, and implementation code.

1 Getting Ready: Stack Overflow

1.1 Problem definition

Stack Overflow is a Q&A website for programmers and developers, irrespective of their expertise level in the domain. It has a wide variety of questions on computer science and programming topics. Registered users can post new questions and answer questions from other users. Each user can collect reputation points. These points are affected by the upvotes or downvotes received by the user on their questions or answers. More reputation points allow users to perform additional functions, like voting to close or delete a question. On achieving reputation milestones, users are awarded badges to highlight their credibility.

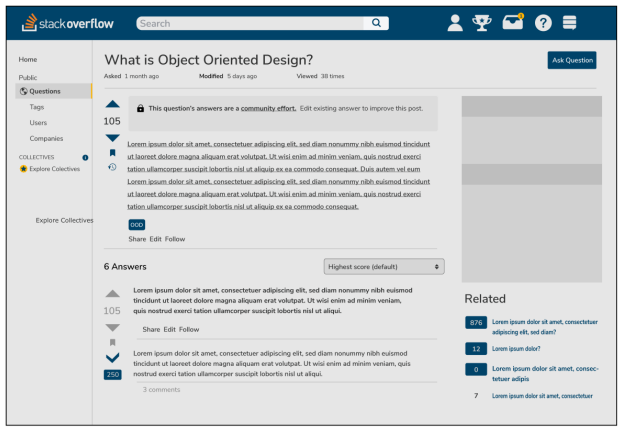


Figure 1.1: A question page on Stack Overflow

1.2 Expectations from the interviewee

It is important to narrow down the components to be included in your Stack Overflow design. The following section provides an overview of some of the main expectations that the interviewer will want to hear you discuss in more detail during the interview:

Discoverability

You may want to ask the interviewer the following to get a better understanding of how Stack Overflow's discoverability works:

- How are users able to search for questions?
- Is there a way to filter questions using tags or users?

Reputation

Reputation points affect user privileges. Therefore, it's important to ask the interviewer the following questions about reputation points:

- How are reputation points calculated? Do users get points for asking or answering questions?
- How many points are required for users to get moderator access?

Voting

Voting is one of the main features of Stack Overflow. It gives insight into which questions or answers are more popular among users. Make sure to ask the following questions to understand how voting works in Stack Overflow:

- What are the different types of voting allowed on Stack Overflow? Are you allowed to upvote and downvote?
- How does voting work when a question has to be closed and deleted? Which user can vote in such circumstances?

Bounty

Bounty is a special reputation placed on a question that is not being noticed or answered. Any user who answers a bounty question receives reputation points equal to the bounty value. You need to clarify the bounty requirements from the interviewer by asking the following questions:

- How are reputation points awarded on bounty questions?
- When do users start a bounty? How long does a bounty last before expiring?

1.3 Design approach

We'll design Stack Overflow using the bottom-up design approach. For this purpose, we'll follow the steps below:

- Identify and design the smallest components first, like a question and an answer.
- Use these small components to design additional components, for example, comments, bounty, and tags.
- Repeat the steps above until we design the complete Stack Overflow platform.

1.4 Design pattern

During an interview, discussing the design patterns that Stack Overflow falls under is always a good practice. Stating the design patterns gives the interviewer a positive impression and shows that the interviewee is well-versed in the advanced concepts of object-oriented design. Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking, "Define design patterns."

Question:

Which design pattern(s) should be used to design Stack Overflow? Please elaborate on your choice(s).

Let's explore the requirements of Stack Overflow in the next lesson.

2 Requirements for Stack Overflow

In this lesson, we'll list Stack Overflow's requirements. This is a crucial step, as requirements define the scope of a problem. Getting them right from the interviewer and understanding them well will make the design of the rest of the system smooth and easy.
We'll use the notational convention to identify each requirement with a unique label "Rn," where "R" is short for requirement and "n" is a natural number.

2.1 Requirement collection

The requirements for Stack Overflow are defined below:

R1: Any guest can view and search questions by tag, username, or word.
R2: Users should be able to add new questions and add answers to an open question.
R3: Users can flag a question, answer, or comment if anything goes against the community guidelines.
R4: A user can upvote, downvote, and add comments to a question or answer, while they can only upvote a comment.
R5: Users can vote to delete or close off questions for community-specific reasons. However, they can only vote to delete an answer.
R6: Users can add a bounty to their question to attract more answers. A Bounty refers to an award of granting reputation points on a question to attract more attention from other users.

R7: Moderators can close or restore a deleted question. Moderators can also delete answers.
R8: The system should notify the user whenever there has been an interaction with them, such as the user's question receiving an answer, earning a badge, or someone upvoting or downvoting their post.
R9: Users can earn badges for their helpful answers or comments.
R10: Users can add tags to their questions. A **tag** is a word or phrase describing the question's topic.
R11: The system should also be able to determine the most popular tags used in questions.

We've identified our requirements for the problem, and in the next lesson, we will define different use cases of Stack Overflow.

3 Use Case Diagram for Stack Overflow

Let's build the Stack Overflow use case diagram and understand its components' relationship. First , we'll define the different elements of our Stack Overflow system, followed by the complete use case diagram.

3.1 System

Our system is Stack Overflow.

3.2 Actors

Now, we'll define the main actors of Stack Overflow.

Primary actors

- **User:** A registered user who can
  - Create, edit, and flag questions and answers.
  - Add bounties and tags.
  - Upvote/downvote content.
  - Accept answers.

  - Add comments.
  - Vote to close/delete questions and answers.

- **Moderator:** A privileged user who can
  - Close, delete, reopen, and restore questions.
  - Delete answers.

Secondary actors

- **Guest:** An unregistered visitor who can
  - Search for and view questions and answers.
  - Register an account.
- **Admin:** A system administrator who can
  - Block or unblock users.
  - Award badges manually.
- **System:** Responsible for
  - Awarding badges based on rules.
  - Sending notifications (on answer, comment, or vote).

3.3 Use cases

In this section, we'll define the use cases for Stack Overflow and list them according to their interactions with a particular actor.

> **Note:** Some use cases will occur multiple times because they are shared among different actors in the system.

User

- **Add/Modify/Flag Question:** Toallow users to post new, update, or report inappropriate questions.
- **Add/Modify/Flag Answer:** Toenable users to contribute answers, edit content, or flag problematic responses.
- **Add Comment:** To let users add clarifying or supporting comments on questions and answers.
- **Upvote a Comment:** To empower users to upvote a comment.

- **Add Bounty:** To let users offer reputation points to attract answers to their questions.
- **Add Tag:** To let users label questions with relevant topic tags for better discoverability.
- **Vote to Close / Delete Question:** To empower users to initiate closure or deletion of low-quality or off-topic questions.
- **Vote to Delete Answer:** To empower users to initiate the deletion of low-quality answers.
- **Upvote / Downvote to Question / Answer:** To enable users to express approval or disapproval of questions and answers.
- **Accept Answer:** To let the authors mark the most helpful answer as accepted.

Guest

- **Search / View Question:** To allow non-logged-in users to browse and search for public questions and answers.
- **Register Account:** To enable guests to sign up and become a full-featured user.

Admin

- **Block / Unblock User:** To enable administrators to restrict or restore user access to the platform.
- **Award Badge:** To allow admins to assign special badges manually to recognize user contributions.

Moderator

- **Close / Reopen / Delete / Restore Question:** To grant moderators authority to manage question visibility and status.
- **Delete Answer:** To allow moderators to permanently remove inappropriate or low-quality answers.

System

- **Send Notification:** To automatically inform users of interactions like votes, comments, answers, or badges.
- **Award Badge:** To automatically grant achievement badges based on predefined contribution criteria.
- **Determine Popular Tags:** To automatically determine the most popular tags.

3.4 Relationships

This section describes the relationships between and among actors and their use cases.

Generalization

- "Moderator" has a generalization relationship with "User" as the moderator can perform all the tasks a normal user can perform.
- "User" has a generalization relationship with "Guest" as the normal user can perform all those tasks that a guest user can perform.

Associations

The table below shows the association relationship between actors and their use cases.

User	Guest	Admin	Moderator	System
Search/view question	Register account	Block/unblock user	Search/view question	Award badge to user
Vote to close/delete question	Search/view question	Award badge	Vote to close/delete question	Send notification
Accept answer			Accept answer	Determine popular tags
Add/modify/flag question			Add/modify/flag question	
Add/modify/flag answer			Add/modify/flag answer	
Add comment			Add comment	
Upvote/downvote question/answer			Upvote/downvote question/answer	
Upvote a comment			Close/reopen/delete/restore question	
Vote to delete answer			Delete answer	
			Upvote a comment	
			Vote to delete answer	

Include

A notification is triggered when a user interacts with content, such as adding an answer or comment, voting, accepting an answer, awarding a badge, or posting a new question.

- The "Send a notification" use case includes the "Add an answer," "Add a comment," "Upvote/Downvote a question/answer," "Accept an answer," "Award a badge," "Upvote a comment," and "Add a question" use cases.

Users can assign a bounty or add tags when posting a new question.

- The "Add a question" use case includes the "Add bounty" and "Add a tag" use cases.

When a user modifies a question, they may also update its bounty or tags.

- The "Modify a question" use case includes the "Add bounty," "Add a tag," "Modify bounty," and "Modify the tag" use cases.

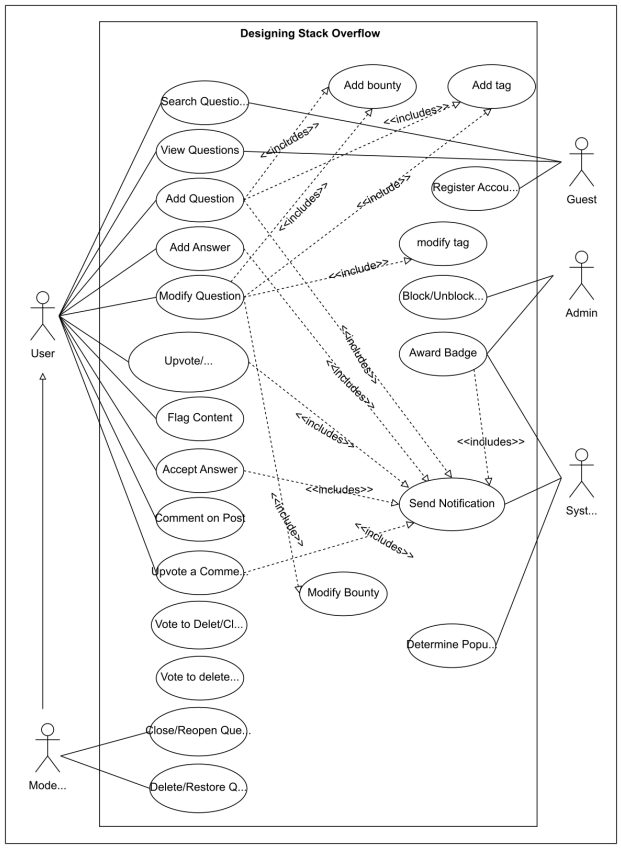


Figure 1.2: The use case diagram for Stack Overflow

3.5 Use case diagram

Here's the use case diagram of Stack Overflow:

In the next lesson, we will discuss the class diagram with a detailed explanation of all classes and their relationship with each other.

4 Class Diagram for Stack Overflow

In this lesson, we'll identify and design the classes, abstract classes, and interfaces based on the requirements we have previously gathered from the interviewer in our Stack Overflow system.

4.1 Components of Stack Overflow

As mentioned earlier, we will design the Stack Overflow system using a bottom-up approach.

Guest

The `Guest` class is a user who can only search and view questions and their answers.
The visual representation of the `Guest` class is provided below:

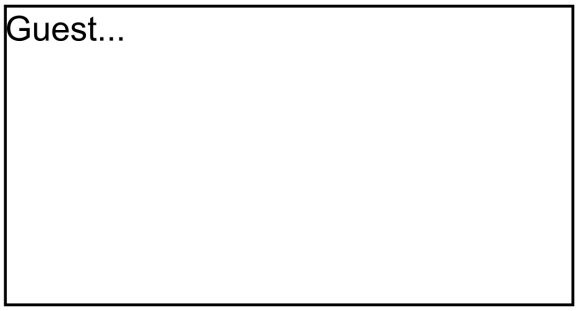


Figure 1.3: The class diagram of the Guest class

R1: Any guest can view questions and search questions by tag, username, or words

Question

The `Question` class is used to enter the details of the question being asked by a user, such as its title, content, tags, etc. It can also allow users to add comments and bounties.
Here is what the class definition looks like:

R2: Users should be able to post new questions and add answers to an open question.
R4: A user can upvote, downvote, and add comments to a question or answer, while they can only upvote a comment.

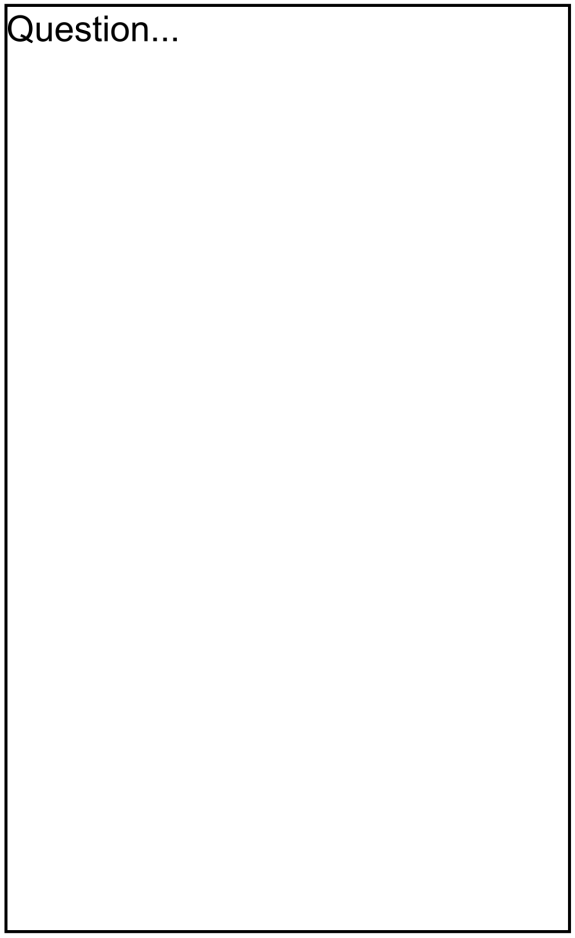


Figure 1.4: The class diagram of the Question class

Answer

The `Answer` class represents a user's answer to a question and will contain elements like content, upvotes, and downvotes.
The UML representation of the class is shown below:

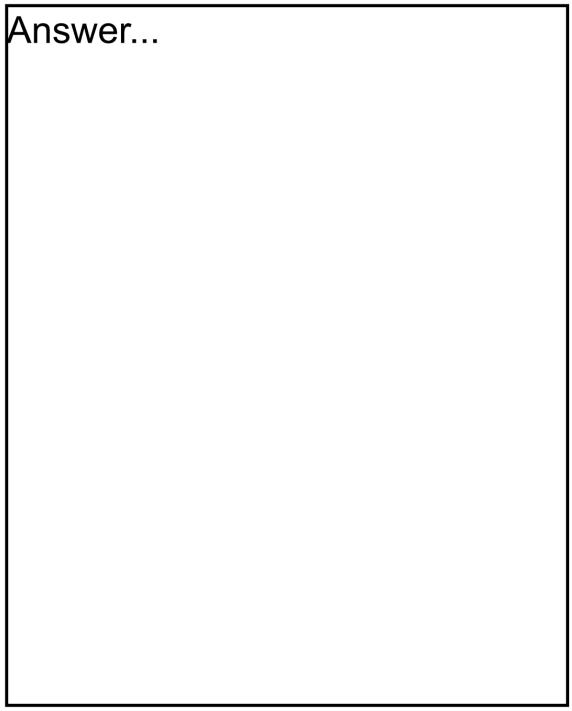


Figure 1.5: The class diagram of the Answer class

R2: Users should be able to post new questions and add answers to open questions.
R4: A user can upvote, downvote, and add comments to a question or answer. However, they can only upvote a comment.

Comment

The `Comment` class refers to an opinion or remark provided by a user on either a question or an answer. It can only be upvoted and flagged, but not downvoted.
The UML representation of the class is shown below:
R4: A user can upvote, downvote, and add comments to a question or answer. However, they can only upvote a comment.

Bounty

The `Bounty` class refers to an award of granting reputation points on a question to attract more attention from other users. It will have an expiry date of seven days and must stay on for at least one day.
The class diagram of the `Bounty` class is shown below:

Figure 1.6: The class diagram of the Comment class

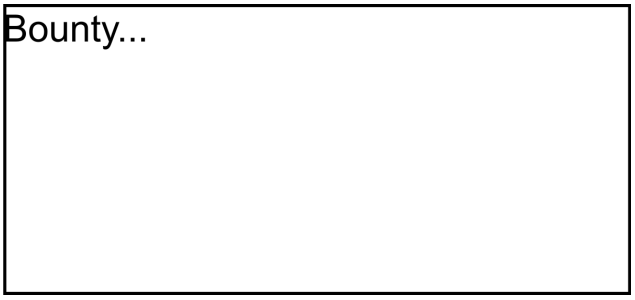


Figure 1.7: The class diagram of the Bounty class

R6: Users can add a bounty to their question to attract more answers.

Badge

The `Badge` class refers to badges on a user's profile that show a reputation-worthy user. The class diagram of the `Badge` class is shown below:

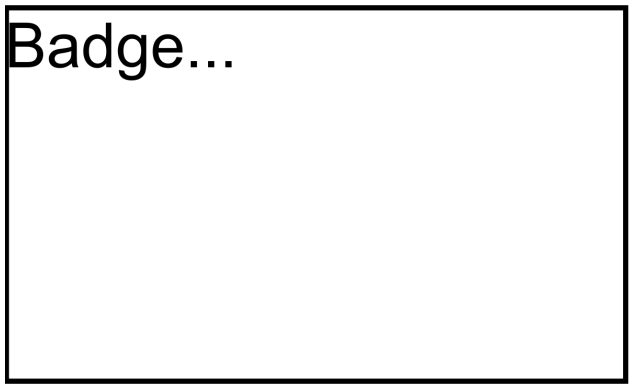


Figure 1.8: The class diagram of the Badge class

R9: Users can earn badges for helpful answers or comments.

Tag and tag list

The `Tag` and `TagList` classes represent the keywords or labels that categorize questions. The `TagList` class contains a key-value pair to keep track of the tag count.
The class diagram for these two classes is provided below:

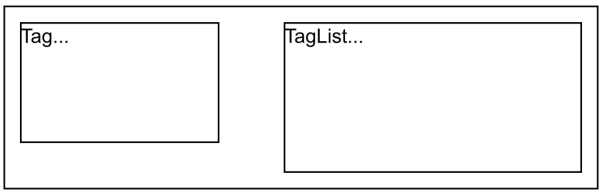


Figure 1.9: The class diagram of the Tag and TagList classes

R10: Users can add tags to their questions. A tag is a word or phrase describing the question's topic.
R11: The system should also be able to determine the most popular tags used in questions.

User

The `User` class is Stack Overflow's main class and is responsible for various operations, such as creating, answering, and flagging questions, voting to delete or close questions, and more. The `User` class is the parent class of the following two classes:

- `Admin`

- `Moderator`

The details of these are provided below:

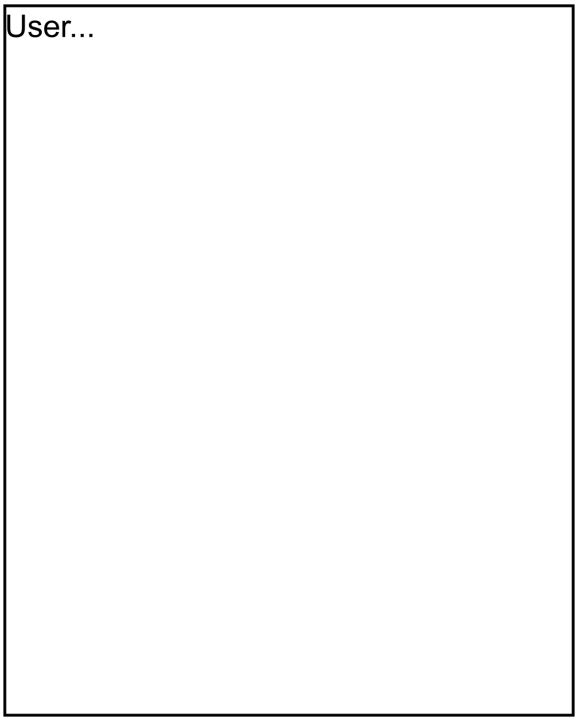


Figure 1.10: The class diagram of the User class

Admin The `Admin` is responsible for performing operations such as blocking or unblocking users.

Moderator The `Moderator` performs operations such as closing, deleting, restoring, and reopening questions.
The class diagram of these classes is provided below:
R2: Users should be able to post new questions and add answers to an open question.
R4: A user can upvote, downvote, and add comments to a question or answer. However, they can only upvote a comment.
R7: Moderators can close questions, restore deleted ones, and delete answers in addition to all the actions available to regular users.

Notification

The `Notification` class will send a notification within the Stack Overflow platform whenever a new answer is added to a question asked by a user.
The UML representation of the class is shown below:

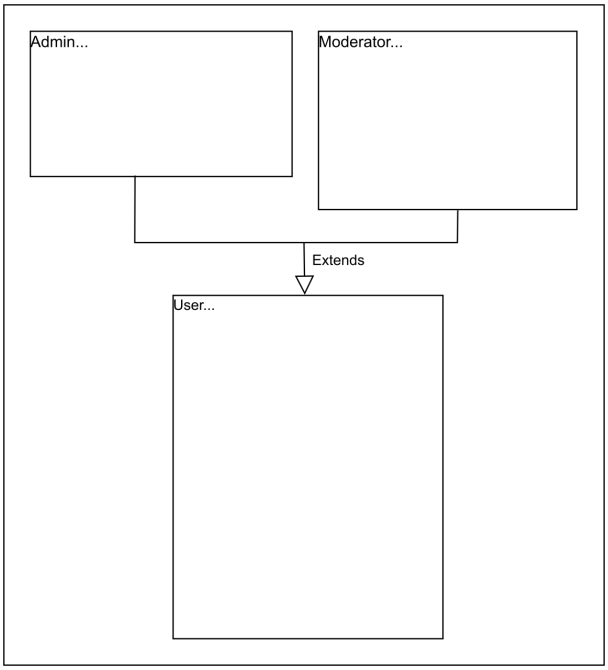


Figure 1.11: The class diagram of User and its child classes

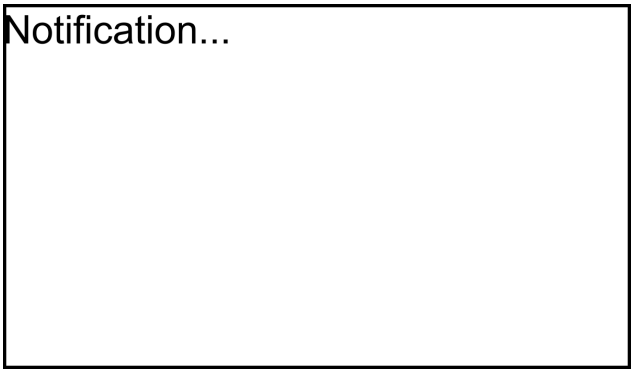


Figure 1.12: The class diagram of the Notification class

R8: The system should notify the user whenever there has been an interaction with them, such as the user's question receiving an answer, earning a badge, or someone upvoting or downvoting their post.

Search interface

The `Search` interface allows users to search for a particular question using tags, usernames, and a string of words.
The UML representation of the interfaces is shown below:

Figure 1.13: The class diagram of the Search interface

R1: Any guest can view and search questions by tag, username, or word.

Search catalog

The `SearchCatalog` is the class where the search functionality is implemented. The catalog will fetch the questions using either tags, usernames, or a string of words.
The class representation of the `SearchCatalog` class is provided below:

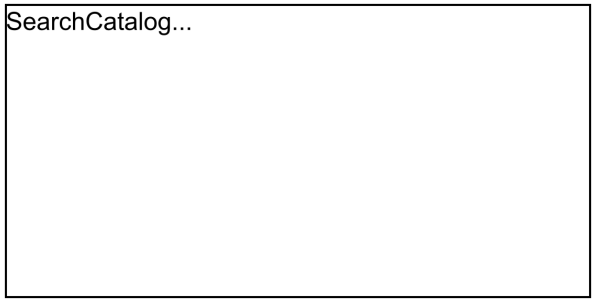


Figure 1.14: The class diagram of the SearchCatalog class

R1: Any guest can view and search questions by tag, username, or word.

Enumerations

The enumerations required in the Stack Overflow design are provided below:

- `UserStatus:` The user status tells us about a user's status, whether active or disabled.
- `QuestionStatus:` The question status describes the status of an existing question, whether it is still active, closed, flagged, or bountied.

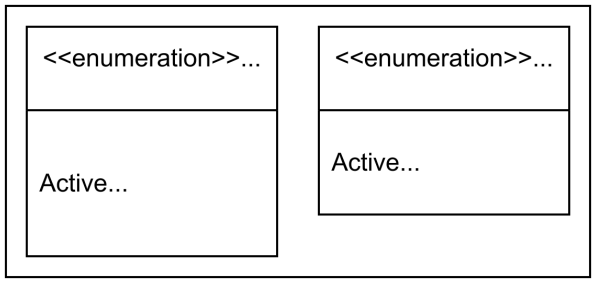


Figure 1.15: Enums in Stack Overflow

4.2 Relationship between the classes

Now, we will discuss the relationships between the classes we have defined above in our Stack Overflow system.

Association

The class diagram has the following association relationships:

- The `User` class has a one-way association with the `Question`, `Comment`, `Badge`, `Notification`, and `Search` classes.
- The `Guest` class has a one-way association with the `Search` class.
- The `Answer`, `Comment`, and `Question` classes have a one-way association with the `Notification` class.
- The `Question` class has a two-way association with the `Tag` class.

Composition

The class diagram has the following composition relationships:

- The `Account` class is composed of the `User` class.
- The `Question` class is composed of the `Bounty`, `Comment`, and `Answer` classes.
- The `TagList` class is composed of the `Tag` class.

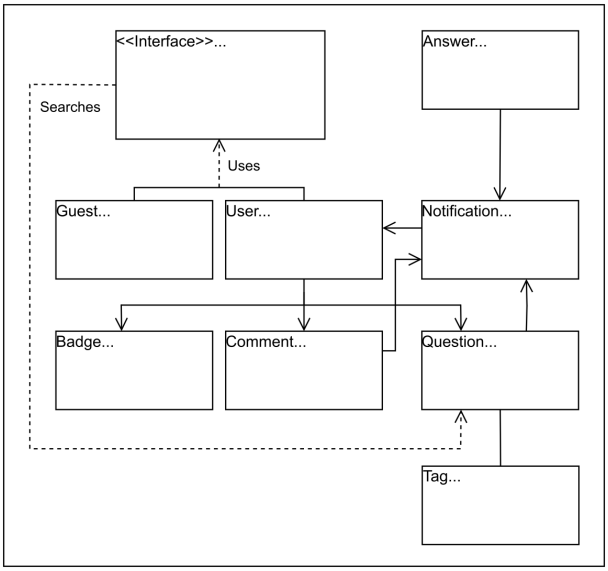


Figure 1.16: The association relationship between classes

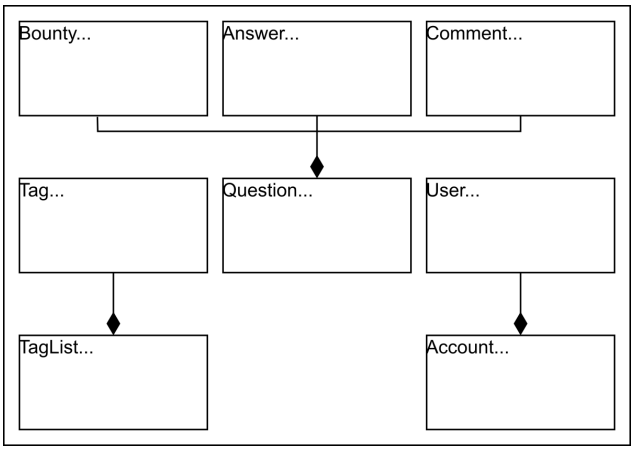


Figure 1.17: The composition relationship between classes

Generalization

The following classes show a generalization relationship:

- The `SearchCatalog` class implements the `Search` interface.

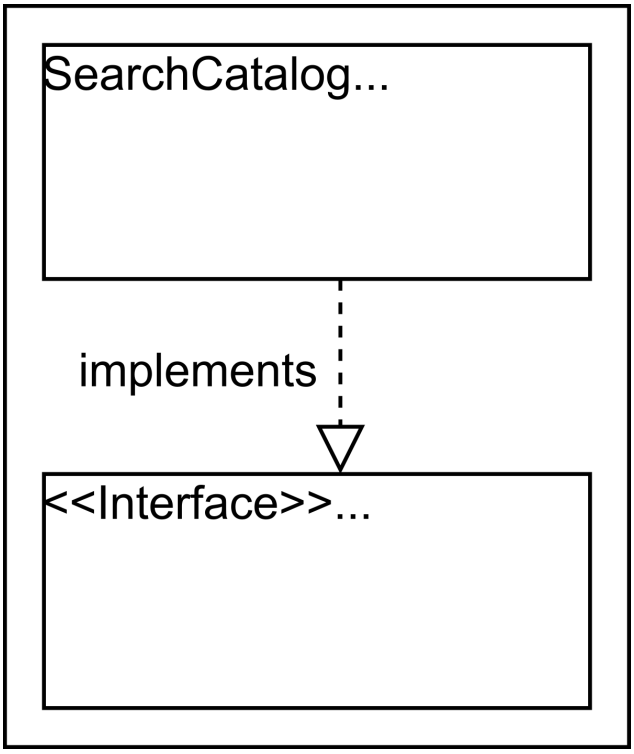


Figure 1.18: The generalization relationship between classes

Inheritance

The following classes show an inheritance relationship:

- Both `Admin` and `Moderator` extend the `User` class.

  Note: We have already discussed the inheritance relationship between classes in the component section above one by one.

4.3 Class diagram of Stack Overflow

In this section, we outline the multiplicity (cardinality) relationships between the main classes in our Stack Overflow system. For each relationship, we explain the allowed number of instances on each side and the real-world or design rationale behind the connection. Understanding these relationships is key to modeling how different entities interact and collaborate to support key workflows in the system.

Here's the complete class diagram for Stack Overflow:

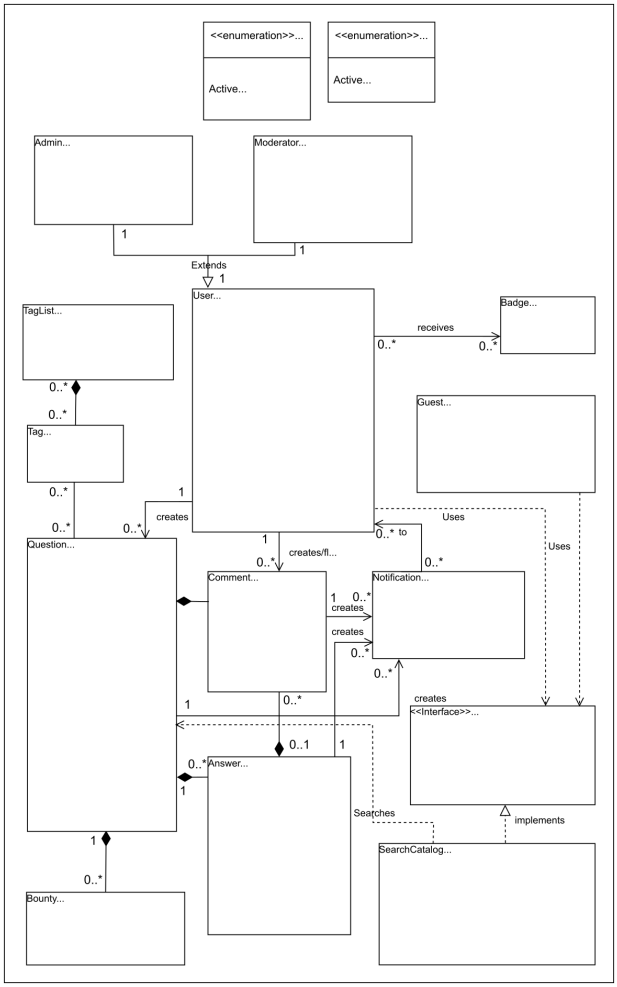


Figure 1.19: The class diagram of Stack Overflow

Source	Target	Multiplicity	Reason
User	Question	1–0..*	A user can create multiple questions.
Question	User	1–1	Each question is created by exactly one user.
User	Answer	1–0..*	A user can provide multiple answers.
Answer	User	1–1	Each answer is associated with exactly one user.
User	Comment	1–0..*	A user can write multiple comments.
Comment	User	1–1	Each comment is posted by one user.
Question	Answer	1–0..*	A question can have multiple answers.
Answer	Question	1–1	Each answer belongs to one question.
Question	Comment	1–0..*	A question can have multiple comments.
Comment	Question	0..1–1	One user posts each comment.
Answer	Comment	1–0..*	An answer can have multiple comments.
Comment	Answer	0..1–1	A comment may be on one answer.
User	Badge	1–0..*	A user can receive multiple badges.
Badge	User	0..*–1	A badge can be awarded to multiple users.
Question	Tag	1–0..*	A question can have multiple tags.
Tag	Question	0..*–1	A tag can be used in multiple questions.
Question	Bounty	0..1–1	A question may have one bounty.
Bounty	Question	1–1	Each bounty is tied to one question.
User	Notification	1–0..*	A user can receive multiple notifications.
Notification	User	1–1	Each notification is sent to one user.

Table 1.1: Entity Relationships and Multiplicities in a Q&A System

4.4 Design pattern

To implement Stack Overflow's core features in a flexible and scalable way, we apply the object-oriented design patterns based on system behavior:

- We know that the system notifies users when actions like adding an answer, comment, vote, or badge occur. To model this behavior, we can use the Observer design pattern.
- Users can search questions using different criteria, such as tags, keywords, or usernames. We can use the strategy design pattern to handle these interchangeable search methods.
- We know the system needs to create different objects, such as questions, answers, comments, and notifications. We can use the Factory Method design pattern to create

these objects flexibly and consistently.

- We know that users like admins and moderators follow a general workflow, but with some variations in control (e.g., deleting questions, restoring answers). We can use the Template Method design pattern to manage this customizable workflow.

- The platform allows dynamic extensions of user capabilities, such as awarding badges or blocking users. Using the Decorator design pattern, we can add such features without modifying existing classes.

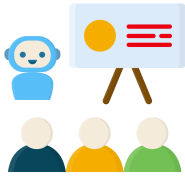
- We know that comments can be added to both questions and answers. We can use the Composite design pattern to treat these comment structures uniformly.

- We know that actions like adding a comment or flagging content could be stored or executed later (e.g., for moderation). We can use the Command design pattern to encapsulate such operations as objects.

- The system has different user roles (User, Moderator, Admin), each with specific permissions. We can use role-based access control (RBAC) combined with inheritance to organize access and shared behaviors.

4.5 AI-powered trainer

At this stage, everything should be clear. If you encounter any confusion or ambiguity, feel free to utilize the interactive AI-enabled widget below to seek clarification. This tool is designed to help you strengthen your understanding of the concepts.



4.6 Additional requirements

The interviewer can introduce some additional requirements in the Stack Overflow system, or they can ask some follow-up questions. Let 's see some examples of additional requirements:

- **Save questions or answers:** Users can save questions or answers and view them later from the "Saved" section in their profile:

We have completed the class diagram of Stack Overflow according to the requirements. In the next lesson, let's design the sequence diagram of Stack Overflow.

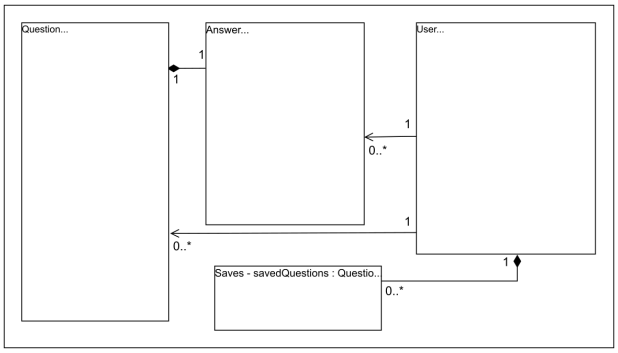


Figure 1.20: The class diagram of the save functionality in Stack Overflow

5 Sequence Diagram for Stack Overflow

Sequence diagrams are a great way to understand the interactions between different entities and objects in the system. There can be different sequence diagrams that we can create for Stack Overflow. In this lesson, we will create sequence diagrams for the following two interactions:

- **Close question:** A user votes to close a question authored by a different user.
- **Sequence challenge:** A user creates a question and adds a bounty.

5.1 Close question

The sequence diagram for closing a question should have the following actors and objects that will interact with each other:

- **Actors:** `User` and `Author`
- **Object:** `Question`

Here are the steps in the closed-question interaction:

1. A user votes to close a question with some remark.
2. If the user is a moderator:
   (a) The question is closed.
   (b) The author of the question is notified about the question.
3. Else if they are a regular user:
   (a) If the user's reputation is greater than or equal to 3000:
      i. A close vote is added.

ii. If there are 3 close votes, the question closes, and the author is notified.

(b) Else if the user's reputation is less than 3000:

i. The vote is considered invalid.

Based on the order above, the sequence diagram for closing a question in Stack Overflow is provided below:

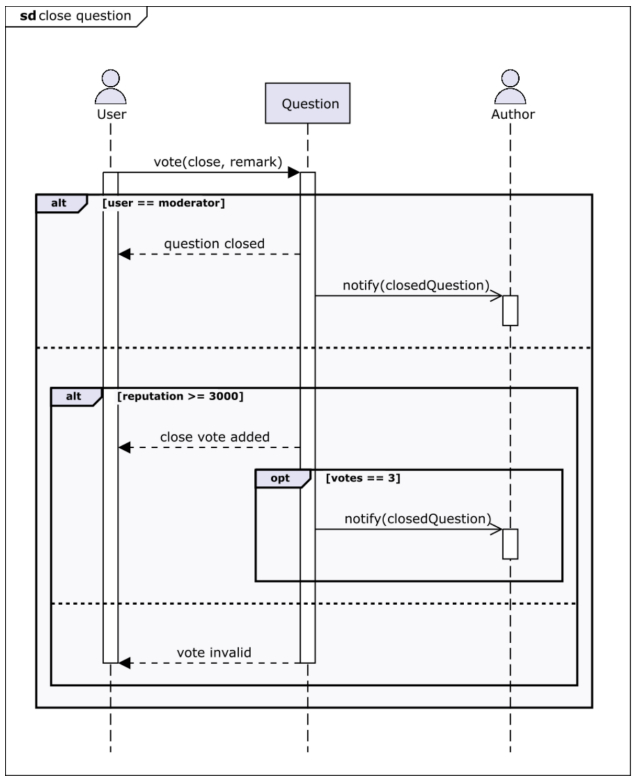


Figure 1.21: The sequence diagram for closing a question

5.2 Sequence challenge: Create question

You'll help us complete a sequence diagram for a new question created by a user where the user adds a bounty to the question after publishing it. A skeleton of the create question sequence diagram is given below:

Notice that the arrows in the diagram above are numbered from 1 to 8. Below are the messages between the actor(s) and object(s). Can you rearrange the messages below in the correct order sequence they should appear in the skeleton of the sequence diagram provided above?

Note: If you're stuck, click the "Show Solution" button.

Alternatively, you can click the "Show complete diagram" button below to see the complete sequence diagram for creating a question interaction.

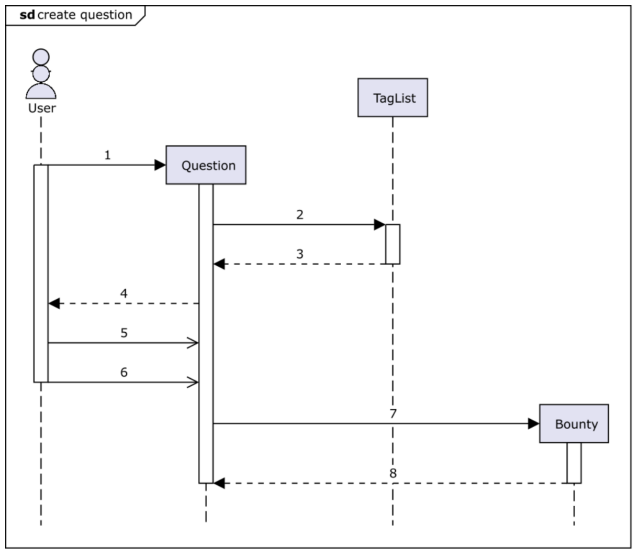


Figure 1.22: The sequence diagram for creating a question

Next, let's examine the activity diagrams for Stack Overflow to understand the system's control flow.

6 Activity Diagram for Stack Overflow

Activity diagrams are a great way to visualize the flow of messages from one activity to another in the system. There can be different activity diagrams that we can create for our Stack Overflow system. In this lesson, we will create activity diagrams for the following two activities:

- A member posting a question.
- **Activity challenge:** A member closes a question.

6.1 A member posting a question

The following are the states and actions that will be involved in this activity diagram.

States

Initial state: The member starts by creating the question..
Final state: There are two final states present in this activity diagram, shown below:

- The question was successfully posted.
- The question is not posted.

Actions

The member creates a question and fills in the required fields. They then add tags to the question. After the post is composed, the system checks the member's status. If the member is a moderator, the question is posted directly. If the member is a regular user, the question will only be posted if they have a good reputation.

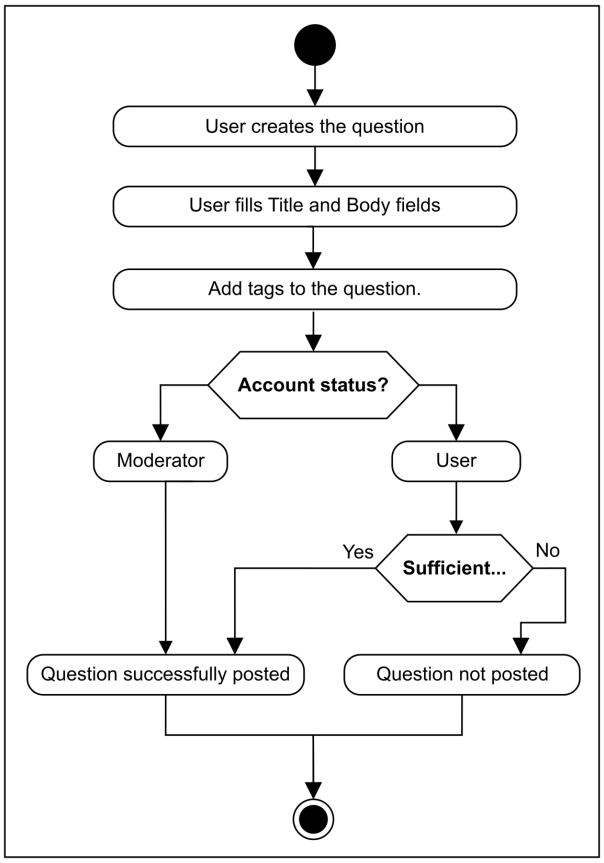


Figure 1.23: The activity diagram for a member posting a question

6.2 Activity challenge: A member closes a question

You'll help us create an activity diagram of a member closing a question. When a question is opened and a user presses the "Close the question," the system checks the account status of the requester. If the requester is a moderator, the question is closed immediately. If the requester is a regular user, the system checks whether they have sufficient reputation. If not, their vote to close is not cast. If the user does have enough reputation, their vote is recorded, and if a total of three or more close votes are gathered, the question is closed. Otherwise , the vote is stored, but the question remains open until the threshold is met.
A skeleton of the activity diagram is provided below:

Notice that the actions in the diagram above are numbered from 1 to 10. The slots below represent the activities, and the arrows represent the flow from one activity to another.

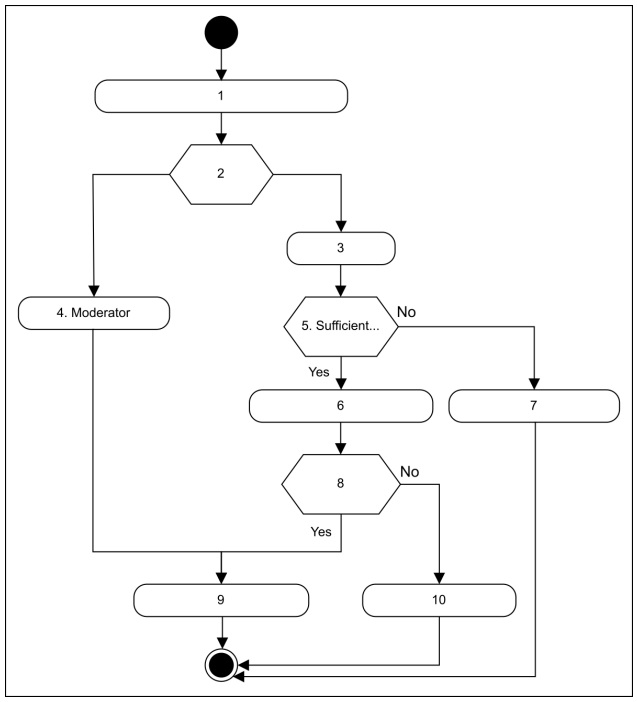


Figure 1.24: The activity diagram for a member closing a question

Can you rearrange the slots below in the correct order they should appear in the activity diagram above?

Note: If you're stuck, click the "Show Solution" button.

Alternatively, you can click the "Show complete diagram" button below to see the complete activity diagram.

We've looked at some of the activity diagrams of our Stack Overflow system. In the next lesson, we will present the code for our designed classes in some of the most popular languages.

7 Code for Stack Overflow

We've gone over the different aspects of Stack Overflow and observed the attributes attached to the problem using various UML diagrams. Let us now explore the more practical side of things, where we will work on implementing the Stack Overflow network using multiple languages. This is usually the last step in an object-oriented design interview process.
We have chosen the following languages to write the skeleton code of the different classes present in Stack Overflow:

- Java
- C#

- Python
- C++
- JavaScript

If you want to skip directly to the implementation and see the complete workflow, simply scroll down or click Executable Code: Designing Stack Overflow.

7.1 Stack Overflow classes

This section will provide the skeleton code of the classes designed in the class diagram lesson.

> **Note:** For simplicity, we are not defining getter and setter functions. The reader can assume that all class attributes are private, accessed through their respective public getter methods, and modified only through their public method functions.

Constants

The following code defines the various enums and custom data types being used in the Stack Overflow design:

> **Note:** JavaScript does not support enumerations, so we will use the `Object.freeze()` method as an alternative that freezes an object and prevents further modifications.

Constant definitions

```
enum QuestionStatus {
  ACTIVE,
  CLOSED,
  FLAGGED,
  BOUNTIED
}

enum UserStatus {
  ACTIVE,
  BLOCKED
}
```

User, admin, moderator, and guest

The `User` class will be a parent class that represents a regular Stack Overflow user. A normal user can also be an `Admin` and a `Moderator`. Another actor is represented by the `Guest` class, which refers to a user who can only search and view questions and their answers. However, they need to register an account to ask or answer questions. The definition of these classes is provided below:

The User, Admin, Moderator, and Guest classes

```
public class User {
  private int reputationPoints;
  private String username;
  private List<Badge> badges;
  private Question currentDraft;

  public Question create(String title, String body, List<Tag> tags);
  public boolean addAnswer(Question, question, Answer answer);
  public boolean createComment(Comment comment);
  public boolean createTag(Tag tag);
  public void flagQuestion(Question question);
  public void flagAnswer(Answer answer);
  public void upvote(int id);
  public void downvote(int id);
  public void voteToCloseQuestion(Question question);
  public void voteToDeleteQuestion(Question question);
  public void acceptAnswer(Answer answer);
  public boolean publishQuestion();
  public void addBounty(int value);
}

public class Admin extends User {
  public boolean blockUser(User user);
  public boolean unblockUser(User user);
  public void awardBadge(User user, Badge badge);
}

public class Moderator extends User {
  public void closeQuestion(Question question);
  public void reopenQuestion(Question question);
  public void deleteQuestion(Question question);
  public void restoreQuestion(Question question);
  public void deleteAnswer(Answer answer);
}

public class Guest {
  public void registerAccount();
  public List<Question> searchQuestions();
  public void viewQuestion(Question question);
}
```

Question, answer, comment, and bounty

Stack Overflow users can create and answer questions, upvote and downvote them, and add bounties and comments to questions. The definition of these classes is provided below:

The Question, Answer, Comment, and Bounty classes

```
public class Question {
  private int id;
  private String title;
  private String content;
```

```
  private User createdBy;
  private QuestionStatus status;
  private int upvotes;
  private int downvotes;
  private int voteCount;
  private Date creationDate;
  private Date modificationDate;
  private Bounty bounty;

  private List<Tag> tags;
  private List<Comment> comments;
  private List<Answer> answers;
  private List<User> followers;

  public void addComment(Comment comment);
  public void addBounty(Bounty bounty);
  public void notify(String closedQuestion);
  public void createBounty(int value);
}

public class Comment {
  private int id;
  private String content;
  private int flagCount;
  private int upvotes;
  private Date creationDate;
  private User postedBy;
}

public class Answer {
  private int id;
  private String content;
  private int flagCount;
  private int voteCount;
  private int upvotes;
  private int downvotes;
  private boolean isAccepted;
  private Date creationTime;
  private User postedBy;
  private List<Comment> comments;

  public void addComment(Comment comment);
}

public class Bounty {
  private int reputationPoints;
  private Date expiryDate;
  public boolean updateReputationPoints(int reputation);
}
```

Badge, tag, and tag list

Users can have badges that act as their reputation awards. Questions can have tags that describe the category that the question falls under. To keep a count of the tags being used, the `TagList` class is used. The definition of these classes is provided below:

The Badge, Tag, and TagList classes

```
public class Badge {
  private String name;
  private String description;
}

public class Tag {
  private String name;
  private String description;
}

public class TagList {
  private HashMap<Tag, int> tagsCount;
  public void incrementTagCount(Tag tag);
  public void decrementTagCount(Tag tag);
}
```

Notification

The `Notification` class is responsible for sending users notifications about new messages, comments, posts, or friend requests via a phone number or email. Its definition is provided below:

The Notification class

```
public class Notification {
  private int notificationId;
  private Date createdOn;
  private String content;

  public boolean sendNotification(User user);
}
```

Search catalog and interface

The `SearchCatalog` class contains information on existing questions and answers. It also implements the `Search` interface class to enable the search functionality based on the given criteria (tags, usernames, and searched keywords). The definition of these two classes is provided below:

The Search interface and the SearchCatalog class

```
public interface Search {
    List<Question> searchByTags(String tagName);
    List<Question> searchByUsers(String username);
    List<Question> searchByWords(String word);
}

public class SearchCatalog implements Search {
    // Map tag names to lists of questions associated with those tags
    private HashMap<Tag, List<Question>> questionsUsingTags;
    // Map usernames to lists of questions created by those users
    private HashMap<User, List<Question>> questionsUsingUsers;
    // Map words to lists of questions containing those words
    private HashMap<String, List<Question>> questionsUsingWords;

    public List<Question> searchByTags(String tagName);
    public List<Question> searchByUsers(String username);
    public List<Question> searchByWords(String word);
    public void indexQuestion(Question question);
}
```

7.2 Executable code: Designing Stack Overflow

Below is a fully self-contained, runnable program in Java, C#, C++, Python, and JavaScript that demonstrates the core workflows of the Designing Stack Overflow system. The system's main driver code resides in `Driver.java`, `Driver.cs`, `Driver.py`, `Driver.cpp`, and `Driver.js` for all respective languages. You can click the "Run" button to execute the code.

What does this code show

- **System initialization:** Initializes a guest account and two users—Alice (`alice123`) and Bob (bob_dev)—with assigned reputation points and active status. This sets up the environment for simulating Q&A interactions on the platform.

- **Scenario 1 (Tag creation and question drafting):** Alice creates two tags, "java" and "oop," and uses them to draft a question about the difference between interfaces and abstract classes in Java. This models how users contribute categorized content to the platform.

- **Scenario 2 (Bounty addition and question publishing):** Alice adds a 100-point reputation bounty to encourage quality responses after drafting the question. She then publishes the question, simulating the transition from draft to public visibility.

- **Scenario 3 (Answer submission and acceptance):** Bob writes an answer to Alice's question, clearly distinguishing interfaces from abstract classes. Alice accepts Bob's answer, showcasing how accepted answers promote helpful contributions within the community.

Hint: Try customizing the code!
This code is designed for experimentation, not just reading. You can edit, extend, or modify any part of the example.

Executable Code: Designing Stack Overflow

```
import java.util.*;

public class Driver {
    public static void main(String[] args) {
        System.out.println("=== Welcome to Stack Overflow Simulation ===
");

        // ================================
        //  System Initialization
        // ================================
        Guest guest = new Guest();
        guest.registerAccount();
        System.out.println("System initialized with a guest account.
");

        // ================================
        //  Scenario 1: User Registration and Tag Creation
        // ================================

        // Creating two users: Alice and Bob
        User user1 = new User();
        user1.setReputationPoints(10);
        user1.setName("alice123");
        user1.setStatus(UserStatus.ACTIVE);

        User user2 = new User();
        user2.setReputationPoints(7);
        user2.setName("bob_dev");
        user2.setStatus(UserStatus.ACTIVE);

        System.out.println("=== Scenario 1: Two users registered. ===");
        System.out.println("- User1: Alice (alice123), Reputation: 10");
        System.out.println("- User2: Bob (bob_dev), Reputation: 7
");

        // Alice creates tags: java and oop
        Tag javaTag = new Tag();
        javaTag.setName("java");
        javaTag.setDescription("Java programming language");

        Tag oopTag = new Tag();
        oopTag.setName("oop");
        oopTag.setDescription("Object-oriented programming");

        List<Tag> tags = new ArrayList<>();
        tags.add(javaTag);
        tags.add(oopTag);
```

```
        System.out.println("Alice created two tags: [java, oop]
");

        // ================================
        //  Scenario 2: Question Creation, Bounty, and Publishing
        // ================================

        Question draftQuestion = user1.create(
            "What is the difference between interface and abstract class in Java?",
            "I am confused about when to use interface and when to use abstract
                class. Can someone explain?",
            tags
        );

        System.out.println("=== Scenario 2: Alice drafted a question: ===");
        System.out.println("- Title: What is the difference between interface and
            abstract class in Java?");
        System.out.println("- Tags: java, oop
");

        user1.addBounty(100);
        System.out.println("Alice added a 100 reputation point bounty to her
            question.
");

        user1.publishQuestion();
        System.out.println("Alice published the question to the platform.
");

        // ================================
        //  Scenario 3: Answering and Accepting the Answer
        // ================================

        Answer answer = new Answer();
        answer.setId(1);
        answer.setContent("Use interface when you want to define a contract. Use
            abstract class when you want base functionality.");
        answer.setPostedBy(user2);

        user2.addAnswer(draftQuestion, answer);
        System.out.println("=== Scenario 3: Bob submitted an answer to Alice's
            question. ===");
        System.out.println("- Answer ID: 1");
        System.out.println("- Content: Use interface when you want to define a
            contract. Use abstract class when you want base functionality.
");

        user1.acceptAnswer(answer);
        System.out.println("Alice accepted Bob's answer as the correct one.
");

        System.out.println("=== End of Simulation ===");
    }
}
```

7.3 Wrapping up

In this chapter, we've explored Stack Overflow's complete design, including how it can be visualized using various UML diagrams and designed using object-oriented principles and design patterns.

8 Summary: Designing Stack Overflow

Now that you've completed the Stack Overflow case study, let's take a moment to reflect on and consolidate what we've learned. We 'll revisit the key system requirements, identify the core classes along with their responsibilities and relationships, and highlight the major design principles applied. We 'll also examine how objects interact within the system and walk through the overall workflow to understand how the components come together to achieve the desired functionality.

8.1 Key requirements

This section outlines the primary functional requirements that shape the design of the Stack Overflow system.

1. Any guest can view and search questions by tag, username, or word.
2. Users should be able to post new questions and add answers to an open question.
3. Users can flag a question, answer, or comment if anything goes against the community guidelines.
4. Users can upvote, downvote, and add comments to a question or answer, but they can only upvote a comment.
5. Users can vote to delete or close off questions for community-specific reasons. However, they can only vote to delete an answer.
6. Users can add a bounty to their question to attract more answers.
7. Moderators can close a question or restore an already deleted question. Moderators can also delete answers.
8. The system should notify the user whenever there has been an interaction with them.
9. Users can earn badges for their helpful answers or comments.
10. The system should also be able to determine the most popular tags used in questions.
11. Users can add tags to their questions.

8.2 System actors

The following actors interact with the Stack Overflow platform in various capacities.

- **User:** A registered member who can post questions, provide answers, comment, vote, and manage their content.
- **Moderator:** A privileged user who can perform all actions of a regular user, plus manage questions (close, delete, reopen) and delete answers.
- **Guest:** An unregistered visitor who can only search for and view questions and answers.
- **Admin:** A system administrator who can manage users (block/unblock) and manually award badges.

8.3 Important classes and relationships

Here are the core classes that form the building blocks of the Stack Overflow system, along with their primary responsibilities and relationships.

8.4 Design highlights

This section covers the strategic decisions made during the design process, including the overall approach, core principles, and specific patterns applied.

Design approach

The system was designed using a bottom-up approach. This methodology begins with designing the smallest, most fundamental components, such as `Question` and `Answer`. These atomic components are progressively combined to build more complex features like `Comment`, `Bounty`, and the entire platform.

Design principles

The design implicitly follows the SOLID principles to create a robust and maintainable system.

- **Single Responsibility principle (SRP):** Each class has a distinct responsibility. For instance, the `Question` class manages question-related data and logic, the `User` class handles user-specific actions, and the `Notification` class is solely responsible for sending notifications.
- **Open/Closed principle (OCP):** The system is open to extension but closed for modification. The use of inheritance, where `Admin` and `Moderator` extend `User`, allows adding new roles with specialized permissions without altering the base `User` class.
- **Liskov Substitution principle (LSP):** Subtypes can be substituted for their base types. A `Moderator` object can be used wherever a `User` object is expected, as it inherits all of `User`'s functionality while extending it.

Class	Description	Important Relationships
Account	Represents a user's account details, including credentials and status.	Composed within the User class.
User	The central class for registered members, handling actions like posting, voting, and commenting.	Associated with Question, Answer, and Comment; extended by Admin and Moderator.
Question	Represents a user-submitted question, containing its content, tags, answers, and status.	Composed of Answer, Comment, and Bounty objects.
Answer	Represents a user-submitted answer to a specific question.	Associated with a Question.
Comment	Represents a comment made on either a Question or an Answer.	Associated with a Question or Answer.
Tag	Represents a keyword or label used to categorize a Question.	Composed within the TagList class; associated with Question.
Bounty	Represents reputation points offered on a Question to attract answers.	Composed within a Question.
Badge	Represents an award given to a User for achieving certain milestones.	Associated with the User class.
Notification	Responsible for creating and sending notifications to users about system events.	Associated with User, Question, and Answer.
Search	An interface that defines the contract for searching questions.	Implemented by SearchCatalog.
SearchCatalog	Implements the Search interface, providing concrete logic to search questions by tags, users, or words.	Implements the Search interface.

Table 1.2: Q&A System Classes and Their Relationships

- **Interface Segregation principle (ISP):** The `Search` interface is a good example, providing a focused contract for search functionality. Clients depend only on this small interface, not the larger concrete `SearchCatalog` class.
- **Dependency Inversion principle (DIP):** The system depends on abstractions, not concretions. High-level components interact with the `Search` interface rather than depending directly on the low-level `SearchCatalog` implementation.

Design patterns

The design leverages established patterns to solve common problems effectively.

- **Observer pattern:** Used to notify users automatically when events occur on content they follow, such as receiving a new answer, vote, or comment.

- **Strategy pattern:** Applied to handle interchangeable search functionalities, allowing users to search for questions by criteria like tags, keywords, or users, without changing the client.
- **Factory Method pattern:** Used to create different objects like questions, answers, and comments, encapsulating the instantiation logic and promoting loose coupling.
- **Template Method pattern:** This pattern defines a general workflow for actions but allows subclasses (like `Moderator` and `Admin`) to customize specific steps, such as deleting a question or restoring an answer.
- **Command pattern:** This pattern encapsulates actions like flagging content or voting as objects, allowing them to be queued, logged, or executed later by a moderation system.

8.5 Object interactions

Collaborations between key objects drive the system's dynamic behavior. The process of closing a question illustrates this well.

- **Vote initiation:** A `User` object initiates the process by calling the vote(close, remark) method on a specific `Question` object.
- **Moderator action:** If the `User` is a `Moderator`, the `Question` object's state is immediately changed to "Closed." It then calls the `notify(closedQuestion)` method, which alerts the `Author` (another `User` object) of the action.
- **Regular user vote:** If the `User` is not a moderator, the `Question` object checks the user's reputation.
- **Vote aggregation:** A close vote is added to the Question object if the user has sufficient reputation. The Question then checks if the total number of close votes has reached the threshold (e.g., 3). If it has, the question's status is changed to "Closed," and the `Author` is notified.
- **Invalid vote:** If the user does not have sufficient reputation, the vote is deemed invalid, and no further action is taken.

8.6 System workflow

The primary workflows demonstrate how actors move through the system to complete tasks. A core workflow is the end-to-end process of a member posting a new question.

- **Initiation:** The workflow begins when a `User` clicks the "Ask Question" button.
- **Content creation:** The `User` fills out the title and body fields for the new question.

- **Tagging:** The system prompts the user to add tags. If the `User` adds a new tag that doesn't exist, the system checks their account status and reputation. A `Moderator` can create a tag directly, while a regular `User` needs sufficient reputation.
- **Policy check:** Once the post is composed, the system validates it against the terms of service.
- **Finalization:** If the post adheres to the terms of service, the question is successfully created and published. Otherwise, the question is not posted, and the user is likely notified of the issue.

9 Key Insights: Stack Overflow

Well done on building your Stack Overflow system design! This lesson shares key insights to help you polish your approach, avoid common mistakes, and easily handle interview questions. You 'll also find a quick quiz to check your understanding and related case studies to boost your confidence.

9.1 Common mistakes

Even strong candidates sometimes slip up when tackling Stack Overflow's design. Watch out for these typical mistakes so you don't lose points during your interview:

- **Overcomplicating the discoverability feature:** Many candidates jump into implementing an overly complex search engine without clarifying that a simple tag, user, or keyword-based search is enough for an MVP. Remember, Stack Overflow's search starts simple, so keep the search logic simple and scalable later. Clearly state that for an MVP, the `SearchCatalog` only needs to support search by tags, usernames, and keywords.
- **Ignoring the reputation system's influence on privileges:** Some forget to link user actions (like voting to close) to reputation thresholds. For example, failing to mention that only users with enough reputation can vote to close questions weakens your answer. Explicitly tie user actions (like voting to close or delete) to reputation levels in your design. Call out thresholds (e.g., minimum 3000 rep for close votes) and show this check in your sequence or class diagrams.
- **Vague handling of notifications:** Candidates often overlook how the notification system uses the Observer pattern. For example, not mentioning how followers of a question get notified when new answers arrive can make your design feel incomplete. Use the Observer pattern for your `Notification` class. Explain that followers of a question or answer are subscribers, and the system automatically notifies them when there's a new answer, comment, or edit.

- **Unclear class responsibilities:** A common slip is mixing up the duties of `User` and `Moderator`. It is clear that `Moderator` extends `User` but adds special privileges like restoring deleted questions.
- **Missing edge cases in voting logic:** Many candidates skip corner cases, like who can downvote a comment or how close votes aggregate before closure. Forgetting this shows gaps in requirement awareness. Always describe vote scenarios fully. Note that comments can only be upvoted, not downvoted. Also, explain how multiple close votes add up and who can trigger them based on reputation or role.

9.2 Key interview tips

Use these focused tips to explain your Stack Overflow design more clearly and confidently handle follow-up questions.

Tip	**Explanation**
Clarify user roles upfront.	State early how Guest, User, Moderator, and Admin relate. Show you understand inheritance and privileges.
Tie actions to reputation.	Always link voting, bounty, and moderation to user reputation levels. This shows awareness of Stack Overflow's trust model.
Leverage design patterns.	Highlight the Observer pattern for notifications to demonstrate practical object-oriented design knowledge.
Use realistic examples.	Back-up features like flagging a question or bounty expiry with a short scenario to prove understanding of real system behavior.
Justify your bottom-up approach.	Explain why you started with classes like Question and Answer, and then composed bigger features. This demonstrates methodical thinking.

Table 1.3: Design and Explanation Tips for a Q&A System

9.3 Test your understanding

These tricky questions help you check if you grasp the Stack Overflow design details and OOD concepts involved.

Stack Overflow

Question 1: What happens next if a question has 2 close votes from users with a reputation of 3000?

- **[CORRECT]** One more close vote is needed.
  Explanation: Three close votes are required for auto-closure by users with sufficient reputation.
- The question closes immediately.
- The question is flagged automatically.
- The question stays open until a moderator acts.

Question 2: Which relationship does `Moderator` have with `User`?

- Association
- Composition
- **[CORRECT]** Inheritance
  Explanation: The `Moderator` class extends the `User` class, so it's an inheritance.
- Dependency

Question 3: Which part of Stack Overflow's design uses composition?

- `User` and `Notification`
- **[CORRECT]** `Account` and `User`
  Explanation: The `Account` class is composed of the `User` class.
- `Moderator` and `User`
- `Guest` and `Search`

Question 4: Which follow-up requirement can change the `SearchCatalog` design later?

- Adding moderator actions.
- Enabling answer deletion.
- **[CORRECT]** Supporting "saved" questions.
  Explanation: Supporting saved questions means indexing them for user-specific retrieval.
- Badge awarding rules.

Question 5: Which scenario best demonstrates the use of an extend relationship in the Stack Overflow use case model?

- **[CORRECT]** A user modifies a question and adds a new tag.
  Explanation: The Modify Question use case extends to Add Tag (or Add Bounty), because adding a tag is an optional extension when modifying a question.
- A guest registers an account to become a user.
- The system sends a notification after an answer is posted.
- A moderator restores a deleted question.

9.4 Related case studies

If you want to broaden your understanding, here are related systems you can practice designing to strengthen your low-level design skills:

- **Quora:** Similar Q&A platform focusing on discoverability, upvotes, and user reputation, but includes following individual users as a core feature.
- **Reddit:** Combines voting and moderation features with nested comments and communities, which is great for practicing user roles and content moderation.
- **Medium:** Publishing platform with comments, upvotes (claps), notifications, and author privileges is useful for comparing different interaction flows.
- **GitHub issues:** A collaborative Q&A/ticketing system with tagging, voting (reactions), and user roles for maintainers and contributors.